

SOFTWARE ENGINEERING (CE304T)

Assignment- 02



SUBMITTED BY:

Usman Raza
Hassan Mehmood
Abdul Tawwab

ROLL # BSCE23047
ROLL # BSCE23031
ROLL # BSCE23054

Jira Project Link:

<https://hassanmef.atlassian.net/jira/software/projects/SCRUM/boards/1>

Project Title: Collaborative Real-time Coding Platform

Part (a) - CLO2

1. Business Use Cases (User Stories) (Story point and priority given in diagrams)

1. As a new user, I want to register using email and password so that I can create a personal account.
2. As a user, I want to log in using Google or GitHub so that I can quickly access the platform without creating a new password.
3. As a user, I want to reset my password via email so that I can recover access if I forget it.
4. As a logged-in user, I want to log out securely so that others cannot access my account on shared devices.
5. As a user, I want to edit my profile including name, avatar, and bio so that my identity is visible to collaborators.
6. As a user, I want to see a list of previously joined rooms so that I can quickly rejoin sessions.
7. As a user, I want to bookmark favorite rooms so that I can access them easily later.
8. As a user, I want to search for rooms by name or ID so that I can find sessions quickly.
9. As a user, I want to create a coding room with a custom name and description so that others know its purpose.
10. As a collaborator, I want to see live edits from others instantly so that we can code together without refreshing.
11. As a collaborator, I want to see each user's cursor with a unique color so that I know who is typing where.
12. As a collaborator, I want to see the name of the user editing a specific line so that contributions are visible.
13. As a collaborator, I want typing indicators so that I know when someone is actively editing.
14. As a collaborator, I want conflict-free editing so that code changes do not overwrite incorrectly.
15. As an admin, I want to remove or ban users so that misuse is prevented.
16. As an admin, I want to monitor active sessions so that system health is maintained.
17. As an admin, I want to log user activity so that suspicious behavior can be tracked.

18. As a user, I want to track performance statistics such as problems solved so that I can measure progress.

2. Architectural Design & Justification

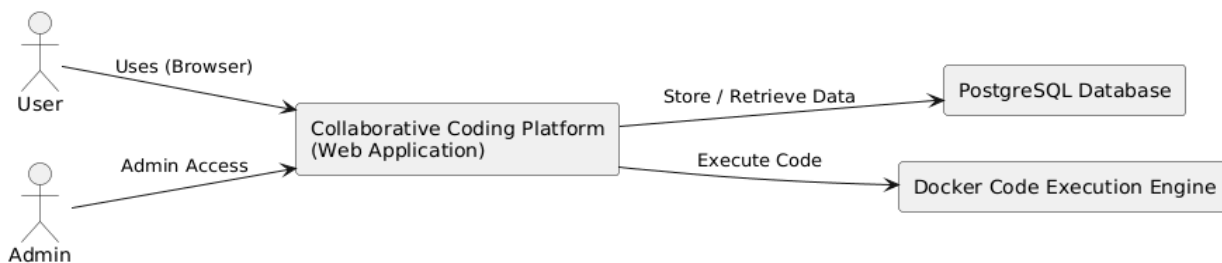
Selected Pattern: Layered Client-Server Architecture

Justification: This pattern is ideal for our client-server web application. It provides a clear separation of concerns:

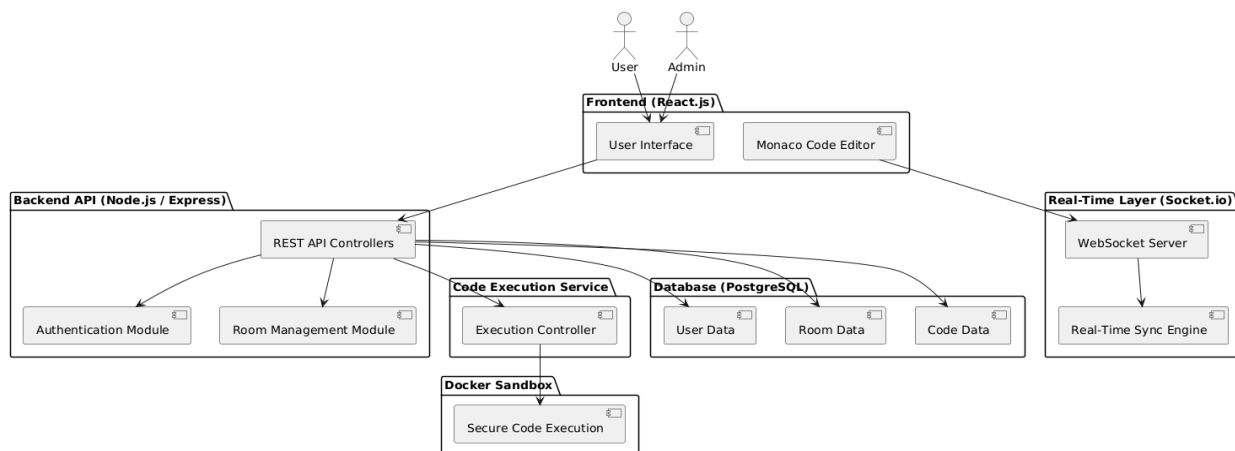
- **Presentation Layer (React.js):** Handles the user interface and Monaco Editor interactions.
- **Application Layer (Node.js/Express):** Manages business logic and APIs.
- **Real-time Layer (Socket.io):** Handles the WebSocket connections for live updates.
- **Data Access Layer (PostgreSQL/Docker):** Manages data persistence and secure code execution in sandboxed environments.

3. Architectural Diagrams

System Context Diagram - Collaborative Coding Platform



High-Level Architecture - Collaborative Coding Platform



Architectural Support:

- **Reusability:** The use of modular React components and well-defined REST APIs allows reuse of UI elements and backend services across multiple features such as coding rooms, chat, and problem selection.
- **Maintainability:** The layered architecture separates concerns, enabling independent updates to components like the database, backend logic, or Docker execution environment without impacting the frontend.
- **Reliability:** The system ensures stable operation by isolating code execution within Docker-based sandboxes and handling real-time communication through WebSockets, reducing the risk of system failure.
- **Security:** Authentication mechanisms (JWT-based) and sandboxed execution environments prevent unauthorized access and protect the system from malicious code execution.

4. Additional Design Diagrams

The following diagrams were created to further illustrate system behavior and design. Due to size and readability constraints, they are provided via an external link:

- Activity Diagram (User Workflow)
- Sequence Diagram (Code Execution Flow)
- Detailed Layered Architecture
- Technology Stack Diagram

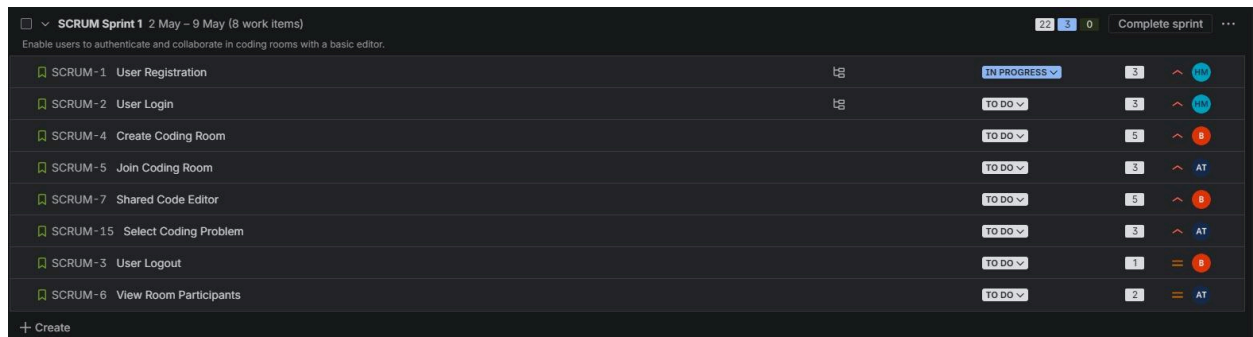
Access Link :- [Link](#)

Part (b) - CLO1: Scrum Management

Sprint 1: Authentication & Real-time Setup

1. Sprint Overview

- **Sprint Name:** Sprint 1 – Authentication & User Management
- **Sprint Duration:** April 30 - May 1
- **Sprint Goal:** Enable users to authenticate and participate in basic collaborative coding sessions, including room creation, joining, and initial code editing. .
- **Jira Board:** Display of task and assigned story points on Jira board. Completed Some of the basic tasks as well



SCRUM Sprint 1 2 May – 9 May (8 work items)

Enable users to authenticate and collaborate in coding rooms with a basic editor.

Task	Assignee	Status	Story Points	Priority
SCRUM-1 User Registration	hg	IN PROGRESS	3	High
SCRUM-2 User Login	hg	TO DO	3	High
SCRUM-4 Create Coding Room		TO DO	5	Medium
SCRUM-5 Join Coding Room		TO DO	3	At Risk
SCRUM-7 Shared Code Editor		TO DO	5	Medium
SCRUM-15 Select Coding Problem		TO DO	3	At Risk
SCRUM-3 User Logout		TO DO	1	Medium
SCRUM-6 View Room Participants		TO DO	2	At Risk

+ Create

Task Distribution:-

Work was distributed among team members based on individual responsibilities and assigned user stories. Evidence of each member's contributions is provided in the following link. [Link](#)

Stand-up Meeting 1 (Mid-Sprint)

Discussion:

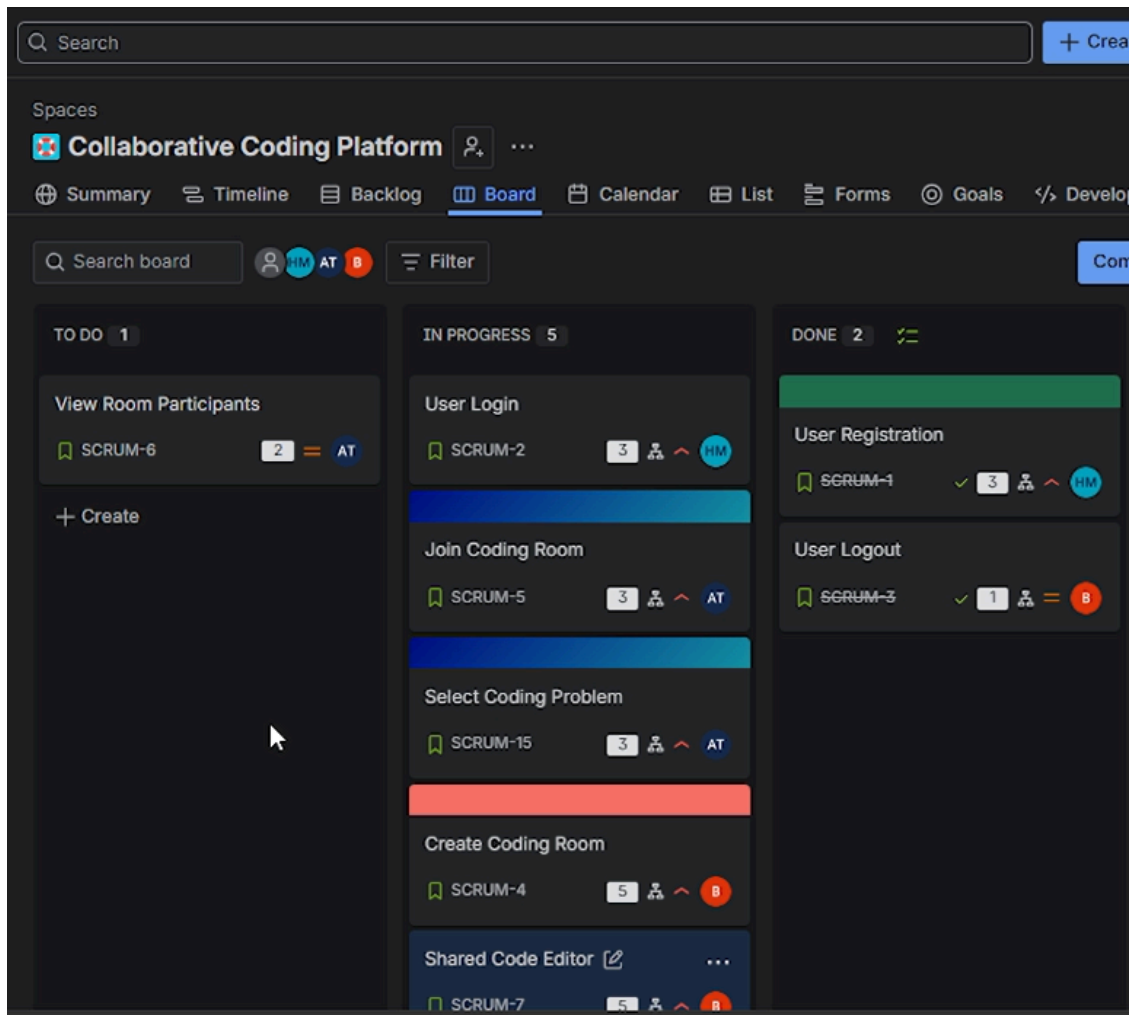
- The team reviewed progress on initial Sprint 1 tasks.
- The authentication module showed steady progress, with user registration completed and login under development.
- The **Join Coding Room feature** was in progress, where the UI had been designed to allow users to enter a room ID and join existing sessions.
- Backend integration for room joining was underway to validate room IDs and ensure proper session entry.
- The shared editor setup had been initiated with basic UI structure.

Issues Identified:

- Challenges in backend integration for room validation and API connectivity.
- Handling invalid or expired room IDs required additional validation logic.

Plan:

- Complete login functionality and backend integration for room joining.
- Improve validation and error handling mechanisms.
- Continue development of core collaboration features.
- **Meeting 1 Link:-** <https://meet.google.com/evq-wcpm-beb>
- **Recording Link 1:-** [Link 1](#)



Sprint 1 Board during Stand-up Meeting 1 showing task distribution across To Do, In Progress, and Done

state

Stand-up Meeting 2 (Almost-End-Sprint)

Discussion:

- Major Sprint 1 features were successfully completed, including user authentication and room collaboration.
- The **Join Coding Room** and **Select Coding Problem** features were fully implemented and functioning correctly.
- Users were able to join rooms using room IDs and select problems to begin collaborative coding sessions.
- The team confirmed that core system functionality was achieved.

Work in Progress:

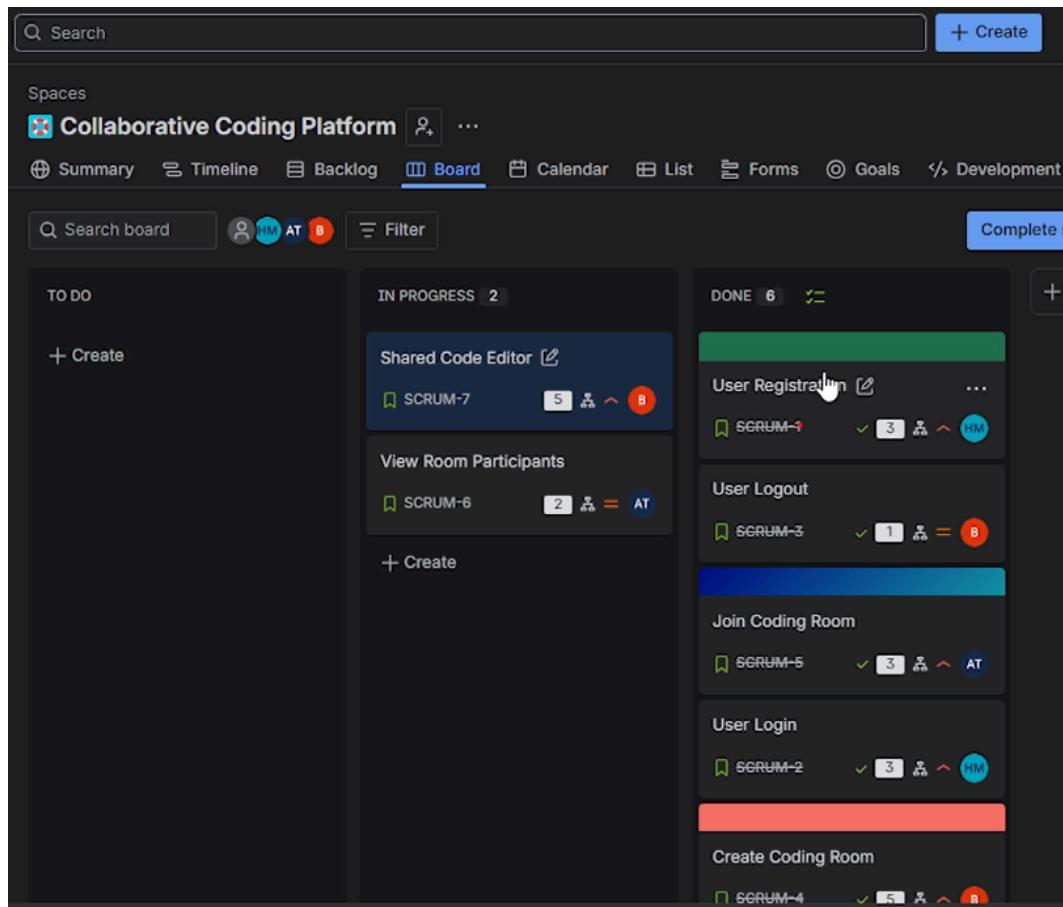
- The **View Room Participants** feature was under development.

Issues Identified:

- A key challenge was handling **real-time updates for participants**, ensuring that when users join or leave a room, the participant list updates instantly for all users.
- Ensuring synchronization without delays or duplication required additional work.

Plan:

- Implement real-time updates for participant tracking using WebSockets.
- Finalize remaining tasks and ensure system stability.
- Prepare for Sprint 2, focusing on advanced features such as real-time synchronization and communication.
- **Meeting 2 Link:-** <https://meet.google.com/xpy-ydae-uub>
- **Recording Link 2:-** [Link 2](#)

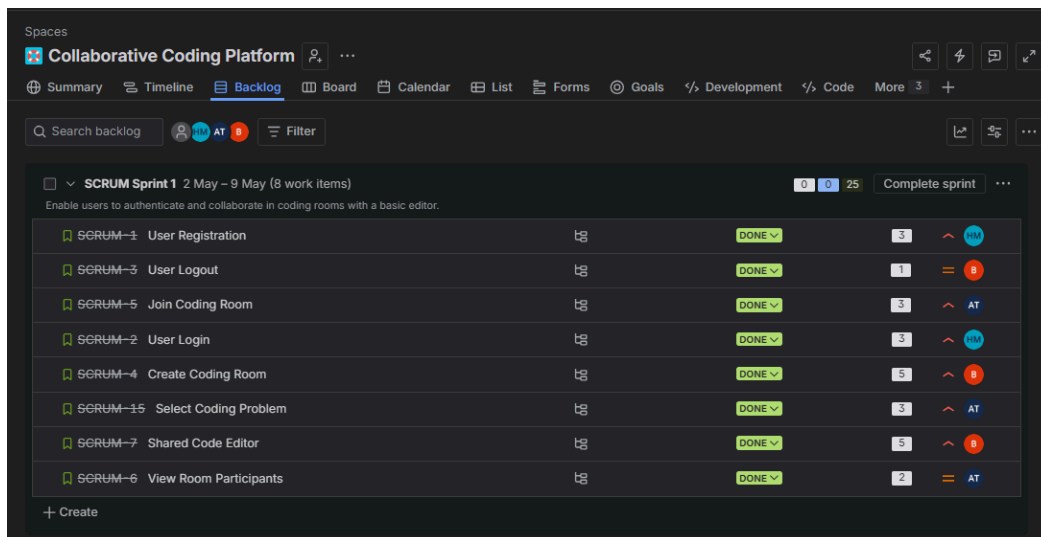


Sprint 1 Board during Stand-up Meeting 2 showing task distribution across To Do, In Progress, and Done state

Sprint 1 Review & Completion :-

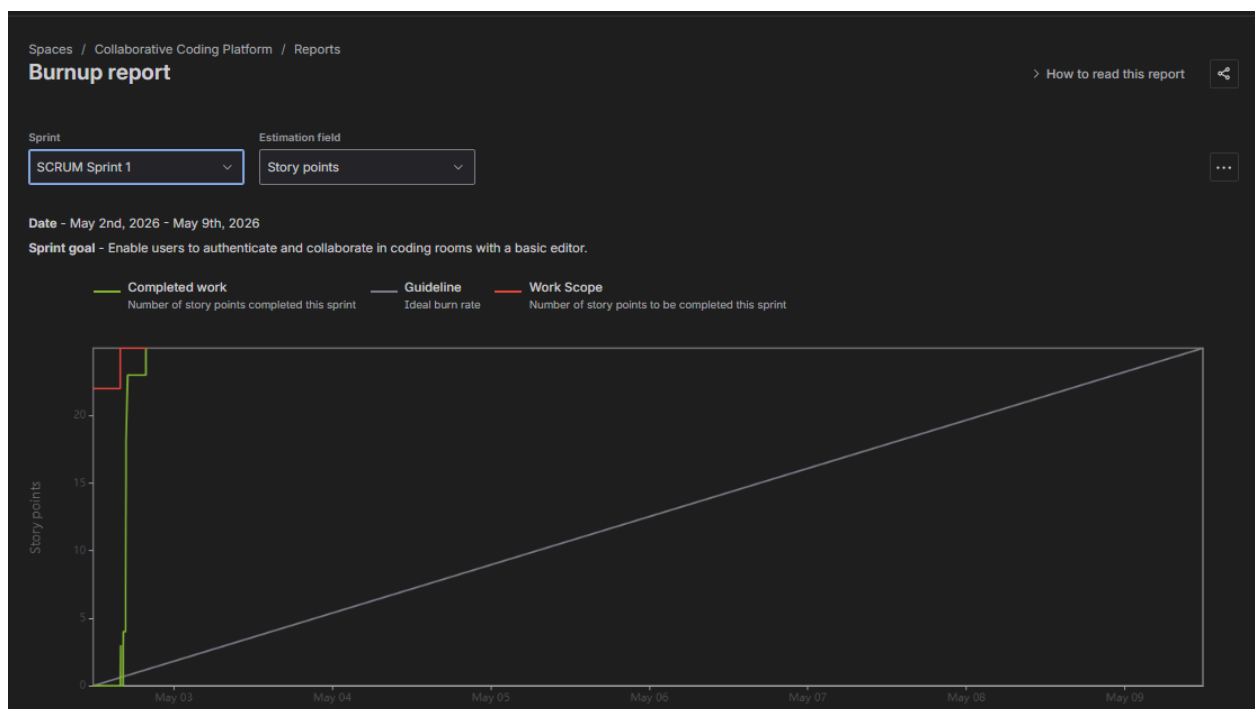
At the end of Sprint 1, all planned user stories and tasks were successfully completed and moved to the “Done” state on the Jira board. Core functionalities including user authentication, room creation/joining, and initial code editor setup were fully implemented.

The team ensured that all features met the sprint goal and were functionally verified before closing the sprint.



Sprint 1 board at completion showing all tasks successfully moved to the Done state.

Burnup Report:



What it shows:

- This chart tracks the **completed work versus total scope** over the sprint duration.

- The **Y-axis** represents story points, while the **X-axis** represents the sprint timeline.
- It includes:
 - **Completed Work:** Progress of finished tasks
 - **Total Scope:** Planned story points for the sprint
 - **Guideline:** Ideal progress trajectory

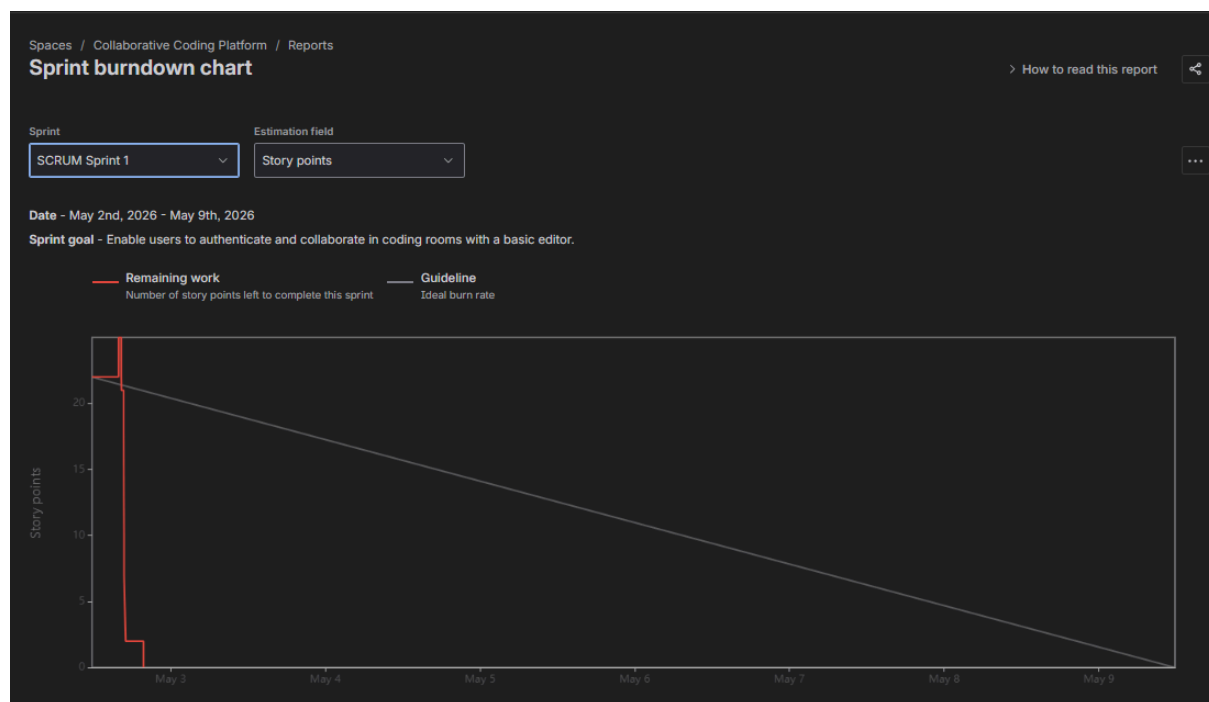
Key Observations:

- The total scope remained mostly stable, indicating that the sprint was well-defined at the start.
- The completed work increased gradually at first and then more rapidly toward the end of the sprint.
- A significant portion of tasks was completed in the later phase of the sprint, reflecting a compressed execution timeline.

Conclusion:

- The sprint goal was successfully achieved, with all planned tasks completed.
- The team demonstrated effective coordination, although task completion was slightly concentrated toward the end.
- In future sprints, a more evenly distributed workload could improve consistency and reduce last-minute pressure.

Burndown Report:



What it shows:

- This chart illustrates the **remaining work (story points)** over the duration of the sprint.
- The **Y-axis** represents remaining work, and the **X-axis** represents the timeline of the sprint.

Key Observations:

- The remaining work decreased steadily as tasks progressed.
- There was a sharper decline toward the end of the sprint, indicating that multiple tasks were completed close to the deadline.
- The remaining work approached zero by the end of the sprint, showing successful completion.

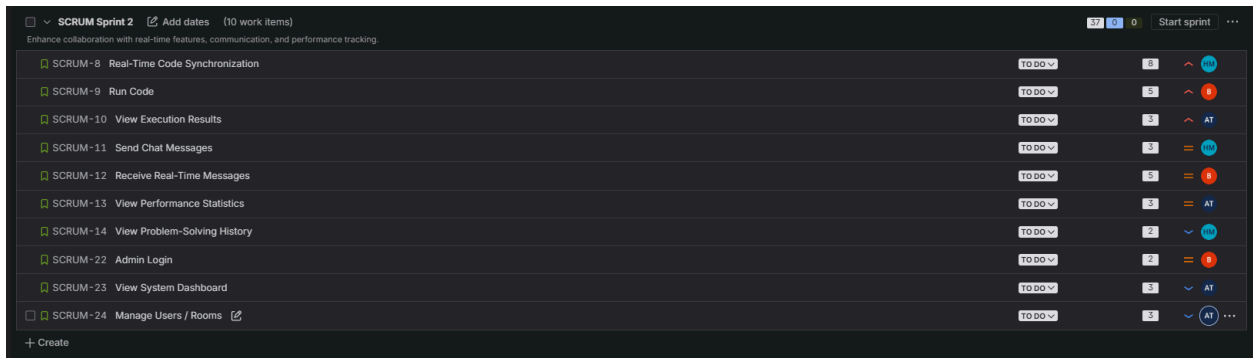
Conclusion:

- The team completed all sprint tasks within the given timeframe.
- While progress was effective, task completion was slightly clustered toward the end of the sprint.
- A more balanced distribution of work could improve efficiency in future iterations.

Sprint 2: Real-Time Collaboration & Execution Features

1. Sprint Overview

- **Sprint Name:** Sprint 2 – Real-Time Collaboration & Execution Features
- **Sprint Duration:** May 2 – May 3
- **Sprint Goal:** Implement Real time code synchronization, view execution results
- **Jira Board:** Display of task and assigned story points on Jira board.



Task Distribution:-

Work was distributed among team members based on individual responsibilities and assigned user stories. Evidence of each member's contributions is provided in the following link. [Link](#)

Stand-up Meeting 1 (Mid-Sprint)

Discussion:

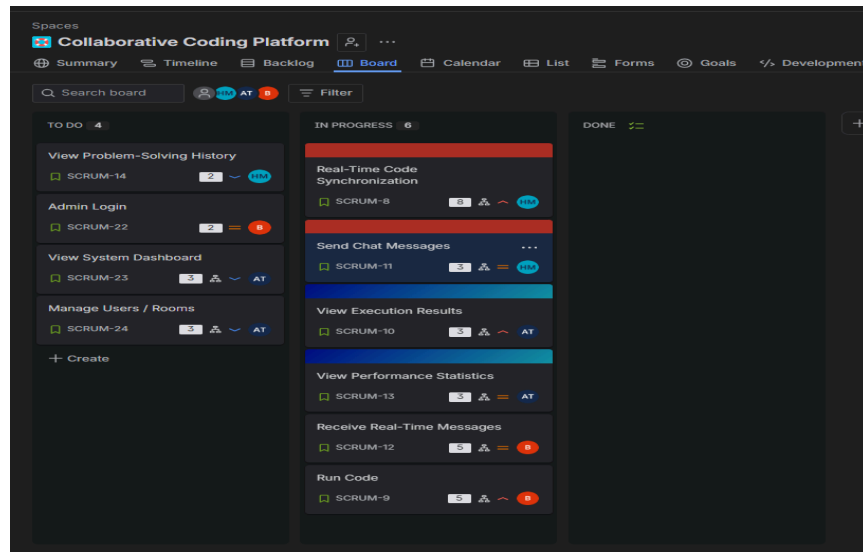
- Real-time code sync architecture completed; backend implementation in progress
- Working on conflict resolution for simultaneous edits
- Chat feature under development with backend + UI integration
- Join Room UI completed; backend integration for room validation in progress
- Real-time messaging & Run Code: UI done, backend in progress
- Admin Login not started yet

Issues Identified:

- Concurrent edits causing overwrite issues in shared editor
- Chat messages facing slight real-time delay
- Room ID validation (invalid/expired IDs) needs improvement
- Real-time communication (WebSockets) not fully stable yet

Plan:

- Continue backend for sync and chat features
- Fix concurrency issues using better WebSocket handling
- Complete Join Room backend integration + validation
- Stabilize real-time messaging and move toward testing
- **Meeting 1 Link:-** <https://meet.google.com/vxa-iwjg-ccs>
- **Recording Link 1:-** [Link 3](#)



Sprint 2 Board during Stand-up Meeting 1 showing task distribution across To Do, In Progress, and Done stat

Stand-up Meeting 2 (Almost-End-Sprint)

Discussion:

- Chat feature fully completed and stable (backend + UI + testing)
- Real-time code sync implemented; currently in testing phase
- Join Room & Select Problem features completed and working
- Run Code and Admin Login features completed
- Problem-Solving History backend done; testing in progress
- View Participants feature under development

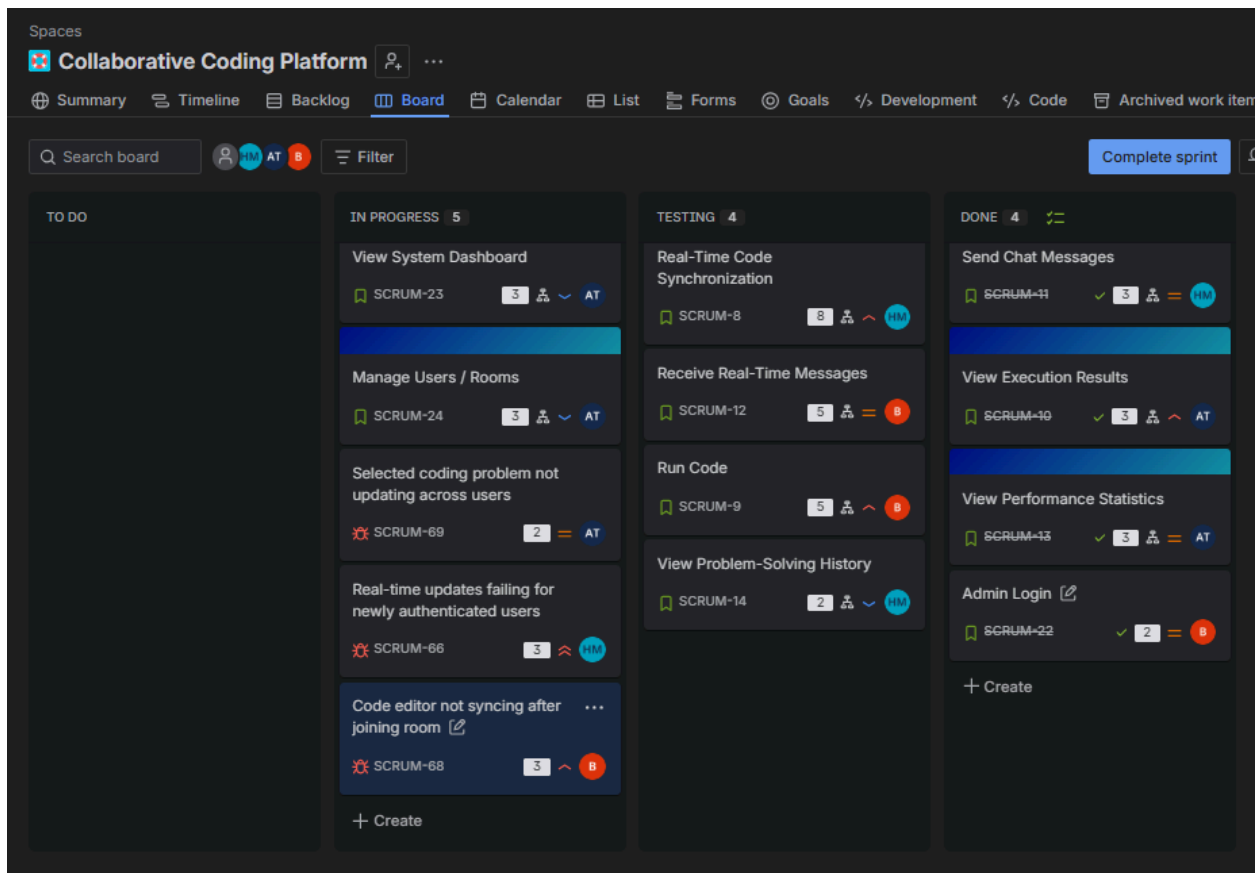
Issues Identified:

- Real-time updates failing after login (WebSocket reconnection issue)
- JWT token not properly attached during WebSocket handshake
- Editor not syncing correctly when new user joins (initial state issue)
- Participant list not updating instantly (real-time sync challenge)

Plan:

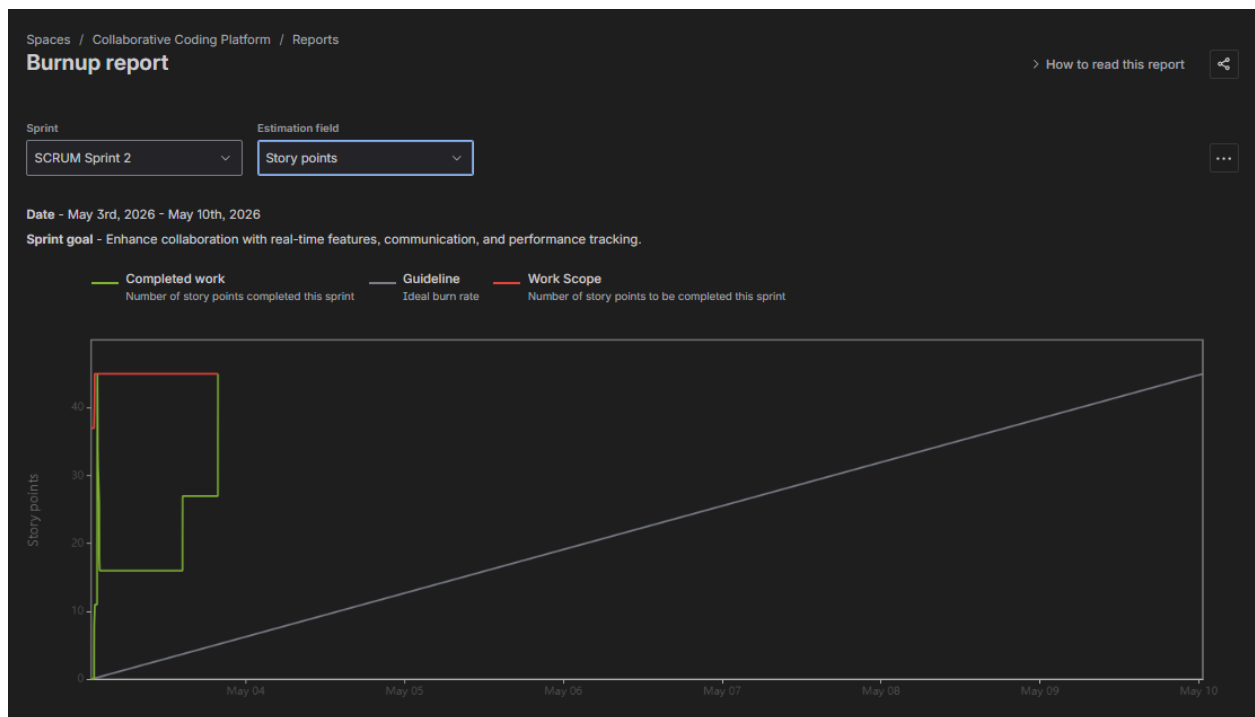
- Fix WebSocket authentication with proper token handling
- Ensure correct state sync when users join rooms

- Implement reliable real-time participant updates
- Perform load testing and finalize system stability
- Resolve remaining issues and close the sprint
- **Meeting 2 Link:-** <https://meet.google.com/evq-wcpm-beb>
- **Recording Link 2:-** [Link 4](#)



Sprint 2 Board during Stand-up Meeting 2 showing task distribution across To Do, In Progress, and Done stat

Burnup Report:



What it shows:

- This chart tracks the **completed work against the total scope** during Sprint 2.
- The **Y-axis** represents story points, while the **X-axis** represents the sprint timeline.
- It includes:
 - **Completed Work:** Tasks and features completed over time
 - **Total Scope:** Planned work including additional bug fixes
 - **Guideline:** Ideal progress trend

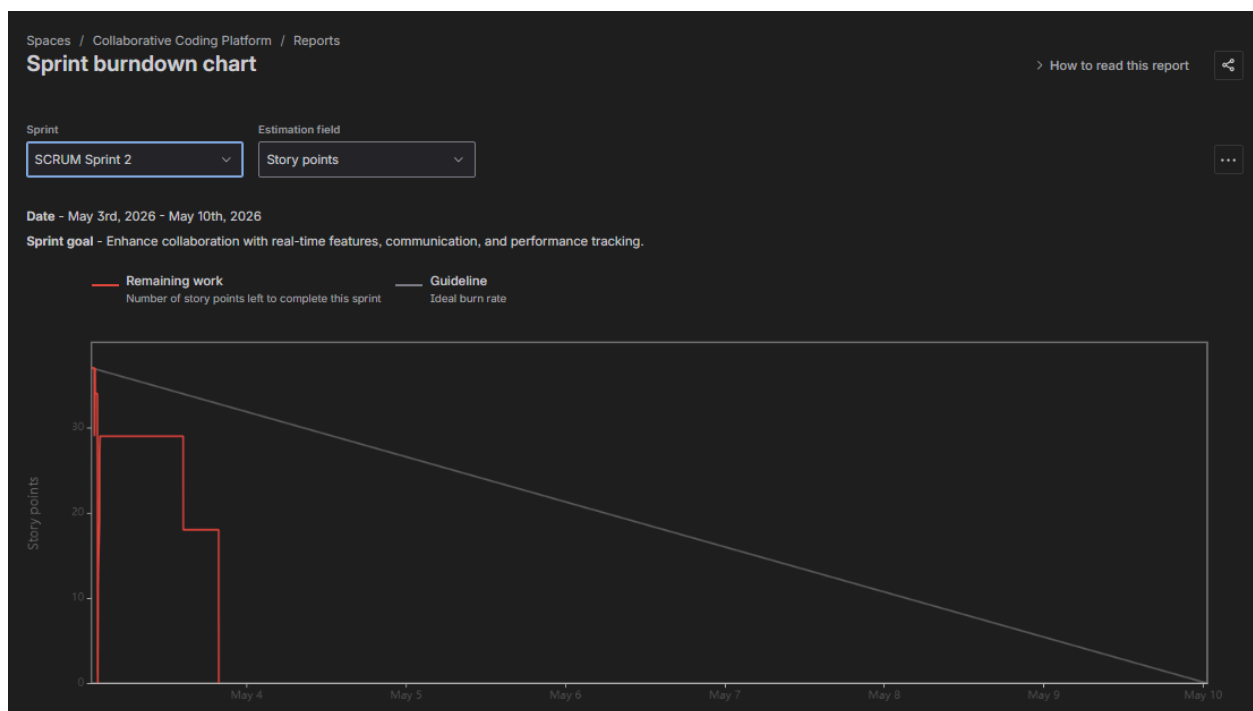
Key Observations:

- The total scope slightly increased during the sprint due to the addition of **bug fixes and integration issues** identified during development.
- Completed work showed steady progress, with a noticeable increase toward the later stages of the sprint.
- Advanced features such as real-time synchronization, chat functionality, and execution services contributed to a heavier workload.
- Final completed work matched the total scope by the end of the sprint.

Conclusion:

- The sprint goal was successfully achieved, including both feature implementation and bug resolution.
- The team effectively handled additional complexity introduced by real-time features and system integration.
- Although some work was completed toward the end, the team maintained control over scope and delivered all planned functionalities.

Burndown Report:



What it shows:

- This chart represents the **remaining work (story points)** throughout Sprint 2.
- The **Y-axis** shows remaining work, and the **X-axis** shows the sprint timeline.

Key Observations:

- The remaining work decreased progressively as development and testing advanced.
- There were sharper drops near the end of the sprint, indicating batch completion of tasks and bug fixes.

- The presence of integration issues slightly affected the ideal trend but did not delay completion.
- The remaining work reached near zero by the end of the sprint.

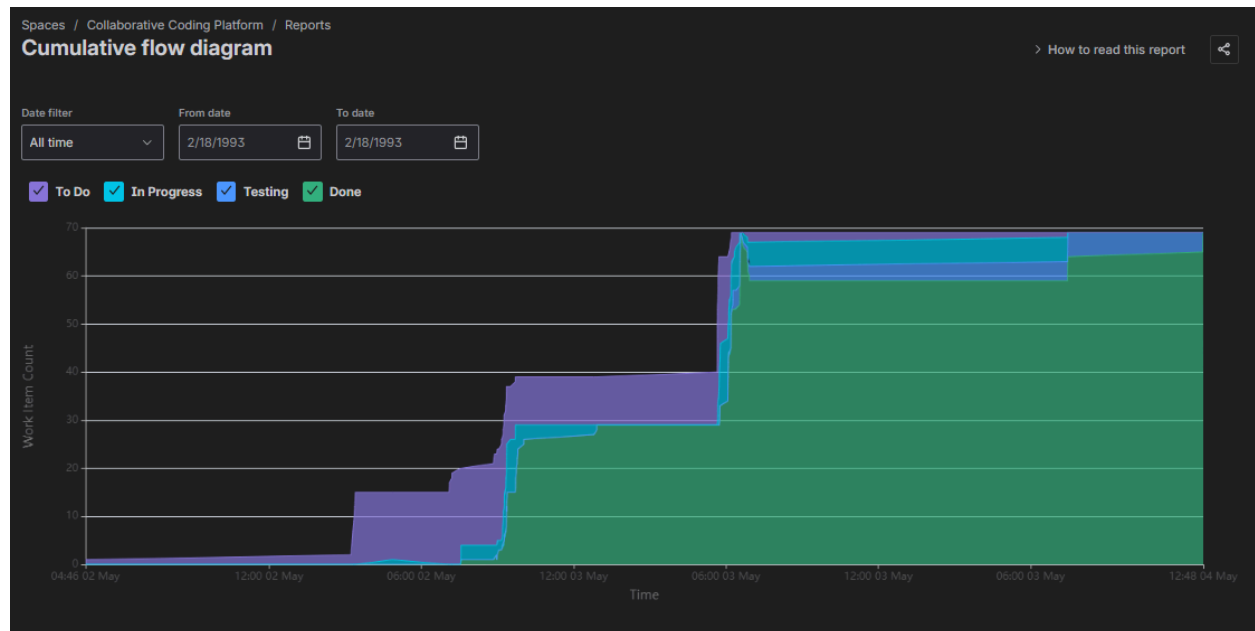
Conclusion:

- All sprint tasks, including advanced features and identified bugs, were successfully completed within the sprint duration.
- The team adapted well to challenges such as real-time synchronization and system integration.
- Future improvements could focus on earlier detection of integration issues to ensure smoother progress throughout the sprint.

Sprint 2 Review & Completion:-

At the end of Sprint 2, all major planned user stories and tasks were successfully completed and transitioned to the “Done” state on the Jira board. Advanced functionalities including real-time code synchronization, chat messaging, run code execution, and admin login were fully implemented and integrated. Core collaboration features were tested under multiple user scenarios, with most synchronization and communication issues resolved.

Cumulative Flow Diagram:



Cumulative Flow Diagram showing task distribution across workflow states during Sprint 2.

Legend:

- Purple → To Do
- Blue → In Progress
- Green → Done

Interpretation:

- **May 2:** Initial phase where most tasks were in the *To Do* state, indicating planning and backlog setup.
- **Early May 3:** A rapid transition of tasks from *To Do* to *In Progress* is observed, showing active development starting.
- **Mid to Late May 3:** Significant movement from *In Progress* to *Done*, indicating high productivity and task completion.
- **May 4:** Most tasks are completed, with the *Done* area dominating the chart and minimal remaining work.

Key Takeaways:

- Tasks progressed efficiently from **To Do** → **In Progress** → **Done** within a short sprint duration.
- Work-in-progress (WIP) was effectively managed, with no major bottlenecks or prolonged delays.
- The team demonstrated strong coordination, completing most tasks within the planned timeline.
- The sharp increase in completed work reflects a **compressed but productive sprint execution**.

Part (c) - CLO4: Jira & Evidence

Links & Deliverables

- **Jira Access Link:**

<https://hassanmef.atlassian.net/jira/software/projects/SCRUM/boards/1>

- **Google Drive (Diagrams/Recordings):**

<https://drive.google.com/drive/folders/1ejWZfbffCxYVjc0T8nH-13aIACw4WITX?usp=sharing>

Project Conclusion

This project successfully demonstrated the application of Scrum methodology in developing a collaborative real-time coding platform. Through iterative sprint planning, daily standups, and continuous progress tracking using Jira, the team was able to deliver both core and advanced system features.

Sprint 1 established the foundational components such as authentication and room management, while Sprint 2 focused on advanced functionalities including real-time synchronization, chat communication, and code execution.

The use of a layered architecture, along with technologies such as React, Node.js, Socket.io, and Docker, ensured a scalable and maintainable system design.

Overall, the project highlights effective teamwork, proper agile practices, and successful implementation of a complex real-time system.